Worksheet No. - 10

Student Name: Sujal Dhiman **UID: 25MCA20186**
Branch: MCA **Section/Group: 1A**
Semester: 2nd **Date of Performance**: 1th Apr, 2026
Subject Name: TECHNICAL TRAINING **Subject Code:25CAP-652**

Aim/Overview of the practical:

To practically apply transaction management concepts and verify how transaction control ensures consistency and integrity.

Objective:
- Implementing transactions with proper syntax.
- Using rollback for error recovery.
- Applying savepoints for partial rollbacks.

Input/Apparatus Used:

- **PostgresSQL**
- **pgAdmin**

Procedure/Algorithm/Code :

```
-- ==========================================
-- PostgreSQL Transaction Experiment
-- ==========================================


-- ========================
-- DROP TABLE (CLEANUP)
-- ========================
DROP TABLE IF EXISTS accounts;


-- ========================
-- CREATE TABLE
```

```
-- ========================
CREATE TABLE accounts (
    id SERIAL PRIMARY KEY,
    name TEXT,
    balance INT
);

-- ========================
-- INSERT INITIAL DATA
-- ========================
INSERT INTO accounts(name, balance) VALUES ('Ankush', 1000);
INSERT INTO accounts(name, balance) VALUES ('Rahul', 2000);

-- View initial state
SELECT * FROM accounts;

-- ==========================================
-- BASIC TRANSACTION (TRANSFER MONEY)
-- ==========================================

BEGIN;  -- Start transaction

-- Deduct 500 from Ankush
UPDATE accounts
SET balance = balance - 500
WHERE name = 'Ankush';
```

```
-- Add 500 to Rahul
UPDATE accounts
SET balance = balance + 500
WHERE name = 'Rahul';

COMMIT; -- Save changes permanently

-- Check result after commit
SELECT * FROM accounts;

-- ==========================================
-- TRANSACTION WITH ROLLBACK (ERROR HANDLING)
-- ==========================================

BEGIN;  -- Start transaction

-- Deduct 300 from Ankush
UPDATE accounts
SET balance = balance - 300
WHERE name = 'Ankush';

-- Simulating an error condition
-- Instead of committing, we rollback
ROLLBACK;

-- Verify that no changes were applied
SELECT * FROM accounts;
```

```
-- ==========================================
-- TRANSACTION WITH SAVEPOINT
-- ==========================================

BEGIN;  -- Start transaction

-- Step 1: Deduct 200 from Ankush
UPDATE accounts
SET balance = balance - 200
WHERE name = 'Ankush';

-- Create a savepoint after first operation
SAVEPOINT sp1;

-- Step 2: Add 200 to Rahul
UPDATE accounts
SET balance = balance + 200
WHERE name = 'Rahul';

-- Suppose something goes wrong here
-- Rollback only to savepoint (undo second step)
ROLLBACK TO sp1;

-- Commit remaining valid changes
COMMIT;
```

```
-- ==========================================

-- FINAL RESULT

-- ==========================================


SELECT * FROM accounts;
```

Output:

```
CREATE TABLE

Query returned successfully in 254 msec.
```

```
INSERT 0 1

Query returned successfully in 58 msec.
```

	id [PK] integer	name text	balance integer
1	1	Ankush	1000
2	2	Rahul	2000

```
BEGIN

Query returned successfully in 64 msec.
```

```
UPDATE 1

Query returned successfully in 57 msec.
```

```
COMMIT

Query returned successfully in 78 msec.
```

	id [PK] integer	name text	balance integer
1	1	Ankush	500
2	2	Rahul	2500

```
ROLLBACK

Query returned successfully in 101 msec.
```

	id [PK] integer	name text	balance integer
1	1	Ankush	500
2	2	Rahul	2500

```
COMMIT

Query returned successfully in 78 msec.
```

	id [PK] integer	name text	balance integer
1	2	Rahul	2500
2	1	Ankush	300

Learning outcomes (What I have learnt):

1. I learned how to use BEGIN, COMMIT, and ROLLBACK to control transactions and ensure that database operations are either fully completed or completely undone.
2. I understood how ROLLBACK helps in error handling by restoring the database to its previous consistent state when something goes wrong.
3. I learned the use of SAVEPOINT to perform partial rollbacks, allowing me to undo specific steps within a transaction without affecting earlier valid operations.
4. I realized the importance of transactions in maintaining data consistency and integrity, especially in real-world scenarios like money transfers between accounts.